

Code Lab 4

Burcu Erarslanoglu (5948355)

EE4C12: Machine Learning for Electrical Engineering Applications
Delft University of Technology

October 3, 2024

1 Task 1: Data Loading & Preparation

For this part, the “load_data” function is first completed. It loads the data from the dataset folder which contains 4 different classes of data. Labels are assigned to these data.

The “fnmatch” function is used in the “load_data” function so that the following labels are assigned.

- Abdomen CT scans - Class 0
- Brain CT scans - Class 1
- Chest X-Rays normal - Class 2
- Chest X-rays with pneumonia - Class 3

Next, it’s time to load our dataset with the “load_data” function. To see that the function works correctly, we verify the sizes as below:

```
1 Number of samples loaded: 400  
2 Number of labels loaded: 400
```

Next, one random image for each class is plotted in the same figure as below:

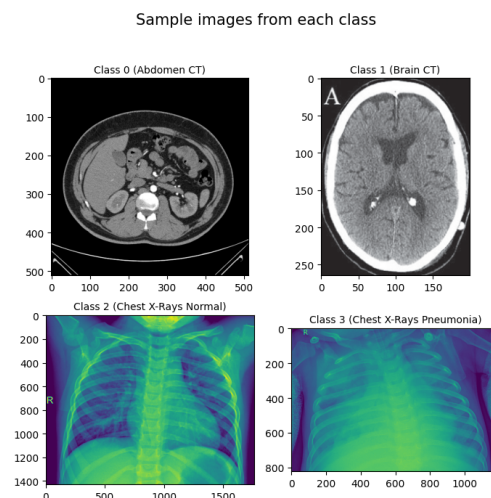


Figure 1: Sample Image from Each Class (Raw)

To make the images uniform, the images are resized and converted to greyscale (the ones that are RGB). For this purpose, the “resize_images_in_list” function is used and inside this one, we

use “convert_to_grayscale”. The final output is by default with size [128,128]. Again, one random image for each class is plotted in the same figure as below:

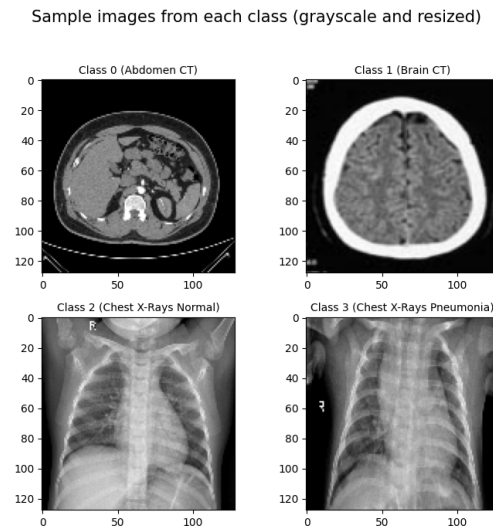


Figure 2: Sample Image from Each Class (Resized and in Grayscale)

The preprocessing steps of resizing and grayscale conversion help standardize the dataset, which is essential for machine learning tasks. In fig. 2, cmap=‘gray’ is used.

2 Task 2: Kernels, Gradients, and Convolution

The function “manual_padding” takes an input matrix and returns the zero-padded version of it. We do this to increase spatial dimension so that after the convolution operations no information is lost. We operate on the following matrix:

```
1 Matrix:
2 [[0 0 0 0 0 0]
3  [0 1 1 1 1 0]
4  [0 1 1 1 1 0]
5  [0 1 1 1 1 0]
6  [0 1 1 1 1 0]
7  [0 0 0 0 0 0]]
```

We zero-pad the matrix as follows:

```
1 Padded Matrix:
2 [[0 0 0 0 0 0 0 0]
3  [0 0 0 0 0 0 0 0]
4  [0 0 1 1 1 1 0 0]
5  [0 0 1 1 1 1 0 0]
6  [0 0 1 1 1 1 0 0]
7  [0 0 1 1 1 1 0 0]
8  [0 0 0 0 0 0 0 0]
9  [0 0 0 0 0 0 0 0]]
```

Pooling is another operation but different from padding, pooling operations are used to reduce the spatial dimensions. As a result, the most important features are retained. The manual discusses two types of pooling max pooling and average pooling. With these methods, the number of features

in an ML model is reduced and this prevents overfitting and improves generalization. By using the padded matrix that was produced, a max pooling operation with a 2x2 window. The result is as follows:

```
1 Max Pooled Matrix:
2 [[0 0 0 0]
3  [0 1 1 0]
4  [0 1 1 0]
5  [0 0 0 0]]
```

Next, we have some given code that defines two Sobel kernels for horizontal and vertical edge detection. To apply these kernels, a “convolution” function is given to us. By using this function with the Sobel kernels, we are able to detect gradients along x and y. In addition to these, we can find the gradient magnitude (absolute value of it) and gradient orientation (converted to degrees). The results of these are presented below:

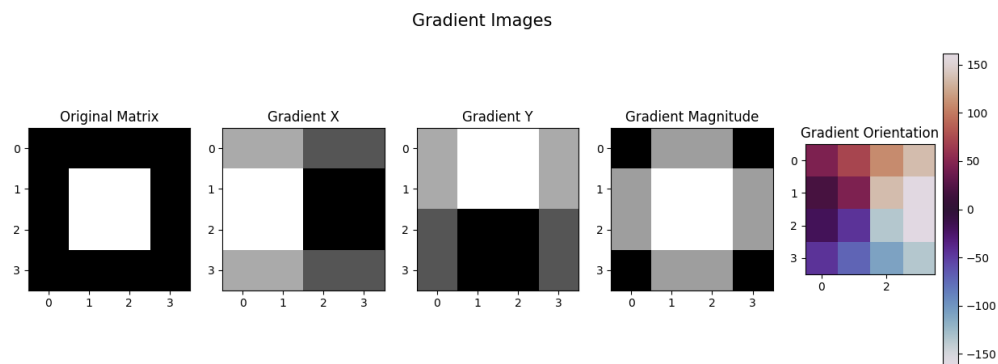


Figure 3: Gradient Images for the Max Pooled Matrix

Using the “mpimg.imread()” function, the “airplane.tiff” image is loaded and the same operations are carried out for this image. The results can be viewed below:

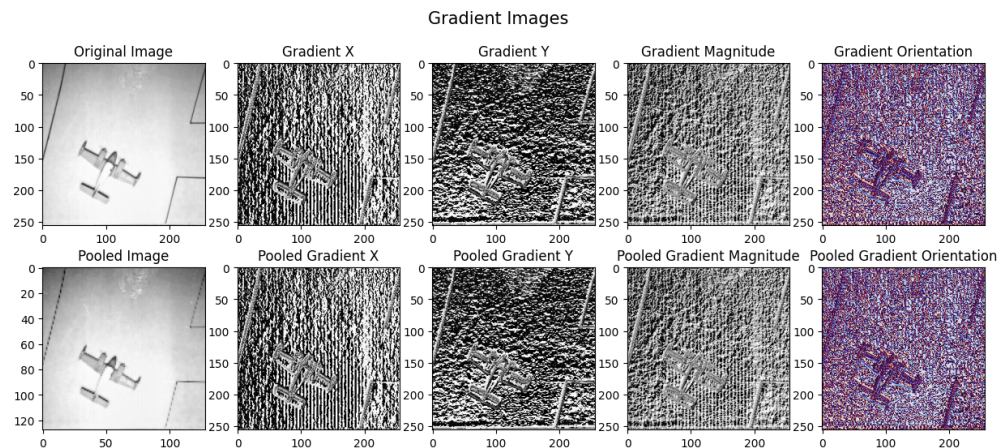


Figure 4: Gradient Images for the Airplane Image

Question 1

Convolution applies a filter which is called the kernel to an image in order to extract features. The features may be edges or textures. This operation retains the image’s size, especially if padding

is applied. On the other hand, pooling focuses on reducing the dimensions of an image so that the most important features can be seen. This process simplifies the data and prevents overfitting. The complexity of the image is reduced and the image's size in the end is less than it was in the beginning.

Question 2

Two separate kernels were used for this purpose because we want to see both the horizontal and the vertical changes (edges) in the image. In other words, horizontal and vertical edges represent changes in different directions, along rows and along columns respectively. A single kernel could be tried but it won't capture both types of edges effectively.

Question 3

The pooled image is a reduced and simplified version of the original. It is smaller and has less detail, but we can still understand that we are looking at an image of a plane so the key features are retained. After pooling, the gradients become sharper along the edges, retaining the key features. Therefore, pooling reduces unnecessary details, such as noises.

Question 4

The resulting images are noisy on the edges because the kernels that were used in this process are very sensitive to high-frequency changes. Small intensity variations are picked up by the Sobel kernels and this creates a noisy image. To prevent this problem, we could use a Gaussian filter which would smooth out the edges. Thresholding is another digital image processing technique that can be used for this purpose.

3 Task 3: Feature Extraction - Histogram of Gradients and PCA

In this section, the Histogram of Oriented Gradients (HOG) and Principal Component Analysis (PCA) are used.

First, the HOG features of the images are obtained using the “get_hog” function. Four combinations, with sizes 16 & 32; and orientations 4 & 8 are used. The 250-th image of each variant of HOG images and the shapes of the respective feature vectors are plotted in the figure below:

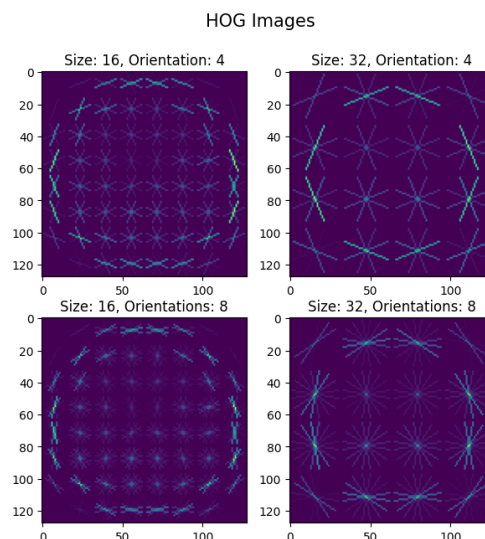


Figure 5: HOG Images

Above in fig. 5, the top row compares HOG images with 4 orientation bins while the bottom row compares HOG images with 8 orientation bin. As the number of orientations increases, the HOG visualizations become more detailed, with finer gradients visible in the bottom row. They are also more densely packed. The next observation is about the cell sizes. The images on the left have smaller cell sizes and the images on the right have larger cell sizes. Smaller cells capture more localized gradient information which are denser. All in all, as cell size decreases and the number of orientation bins increases; the overall complexity increases.

Second, we apply the PCA on the resized grayscale images. The images are reshaped so that PCA can be applied. The scatter plot of the first two components of the PCA transformed features is given below:

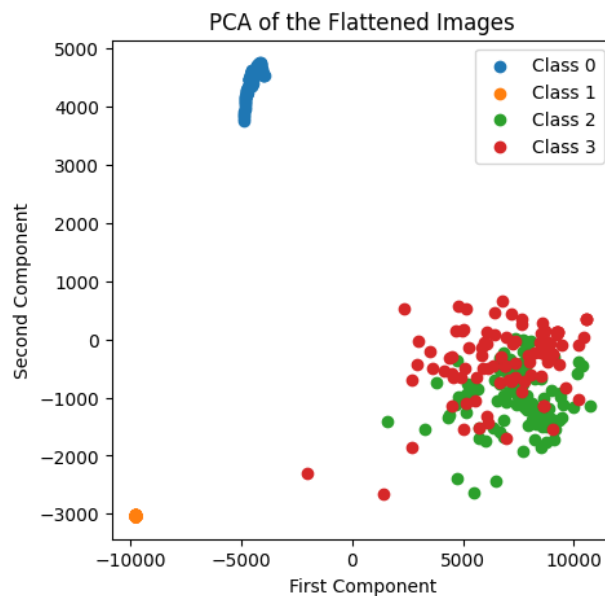


Figure 6: PCA of the Flattened Images (First Component and Second Component)

The explained variance and cumulative explained variance ratios of the first 20 components are also plotted as below:

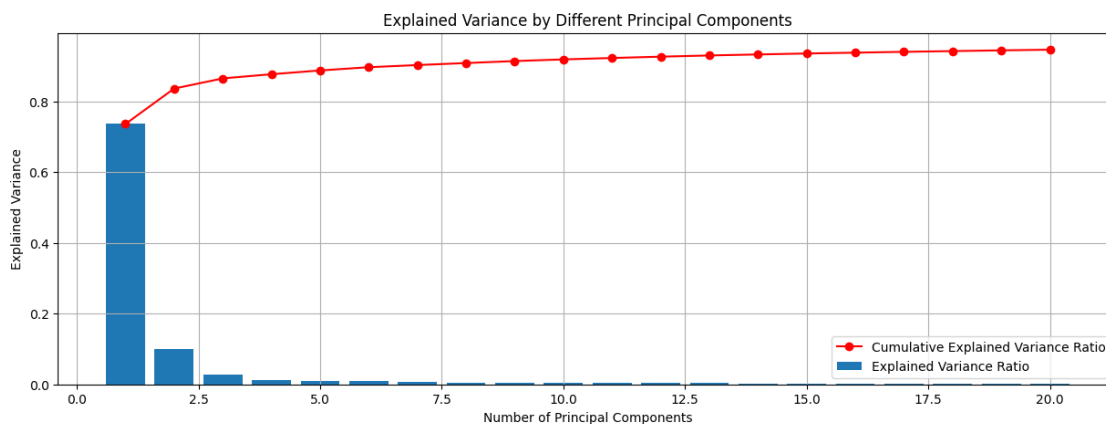


Figure 7: Explained Variance by Different Principal Components

Question 1

The HOG images having 16 pixels per cell have smaller cell sizes and they result in larger feature vectors when compared to the HOG images having 32 pixels per cell. While 16 pixels per cell one is densely packed 32 pixels per cell is sparser. When we compare the HOG images to the original flattened images, the HOG features provide structural and directional information while the original flattened images don't. This makes HOG images suitable for classification tasks.

Question 2

The model performances should improve when HOG features are used over the original flattened images. Usually, HOG features help by providing the gradient and edge information. This is useful in applications where we need to distinguish between classes based on textures. Performance depends on the number of orientation bins and cell sizes as well.

Question 3

The PCA plot of the first two components shows that the classes are mostly distinguishable. For Classes 2 and 3 however, this is slightly harder. Classes 0 and 1 are distinct, forming a separate cluster.

Question 4

Classes 2 (normal chest X-rays) and 3 (chest X-rays with pneumonia) have very similar image structures, which explains the overlapping behavior observed in the PCA plot. They have an overlapping feature space in the reduced dimensional space (first two components). This loss of separability can be solved by adding components.

Question 5

The explained variance represents the amount of information or variance in the data that is captured by a principal component. Components with high explained variance ratios contribute more to describing the data's structure. The first few components account for the majority of the variance, indicating that they contain most of the important information. The later components contribute less, which indicates a diminishing return.

4 Task 4: Model Training

In Task 4, we classify the images based on the extracted features using various ML models. The function "Classifier" is defined to fit the models and compute performance metrics such as Accuracy, Recall, Precision, and F1; while also plotting the confusion matrix. Different classifiers such as K-Nearest Neighbours (KNN), Support Vector Classifier (SVC), and Multi-Layer Perceptron (MLP) are used.

4.1 Original (Resized and then Flattened) Image Dataset

The KNN model is run with the following hyperparameters: $n_{neighbors} = 3, 4, 5$, while the SVC is evaluated with different kernels ('linear', 'poly', 'rbf'). In the case of the 'poly' kernel, the degree is set to 3, and hyperparameters such as $C = 1.0$, $gamma = 'auto'$, and $tol = 1e - 3$ are used. The MLP model is run with different hidden layer sizes (ranging from (100) to (100, 100, 100)), using ReLU activation, Adam solver, and $alpha = 0.0001$. The results obtained for this part are presented below:

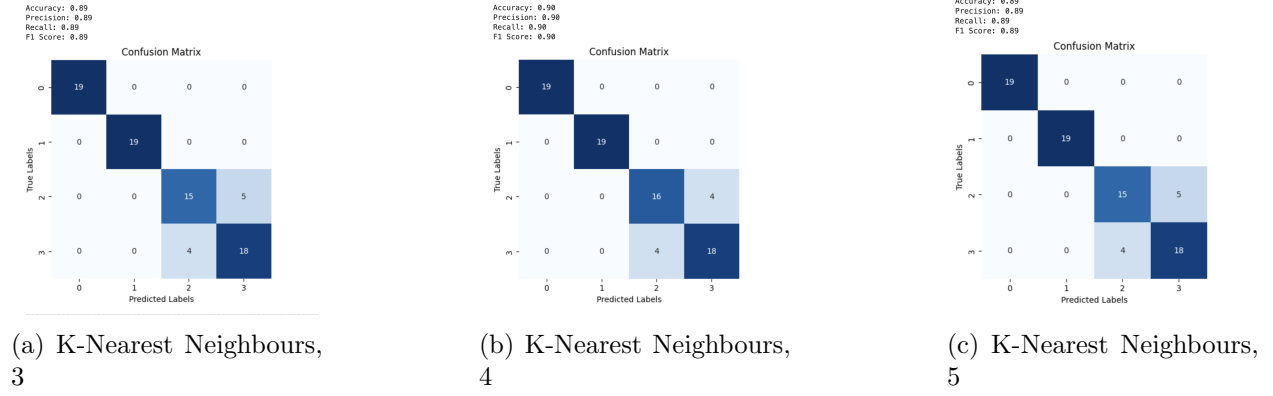


Figure 8: K-Nearest Neighbours Subplots

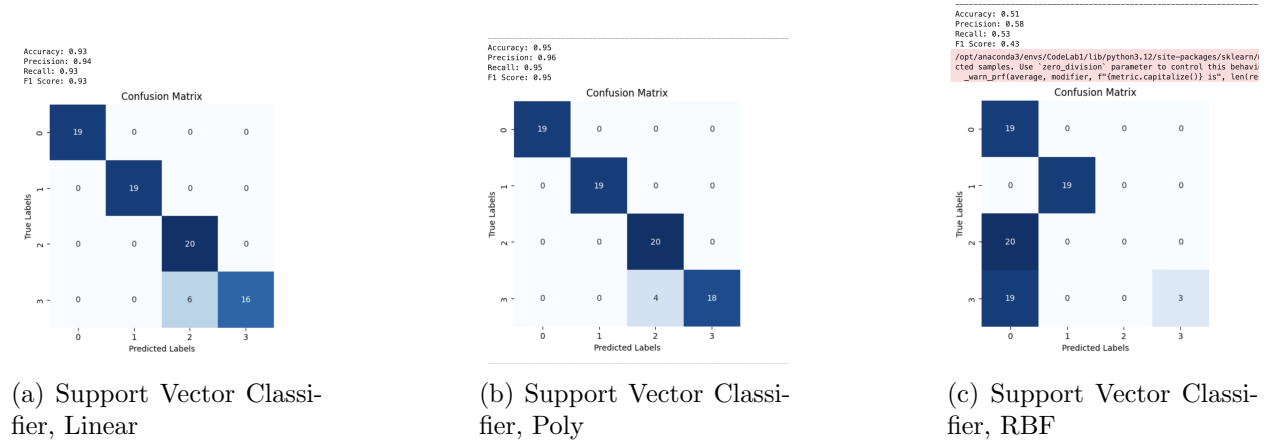


Figure 9: Support Vector Classifier Subplots

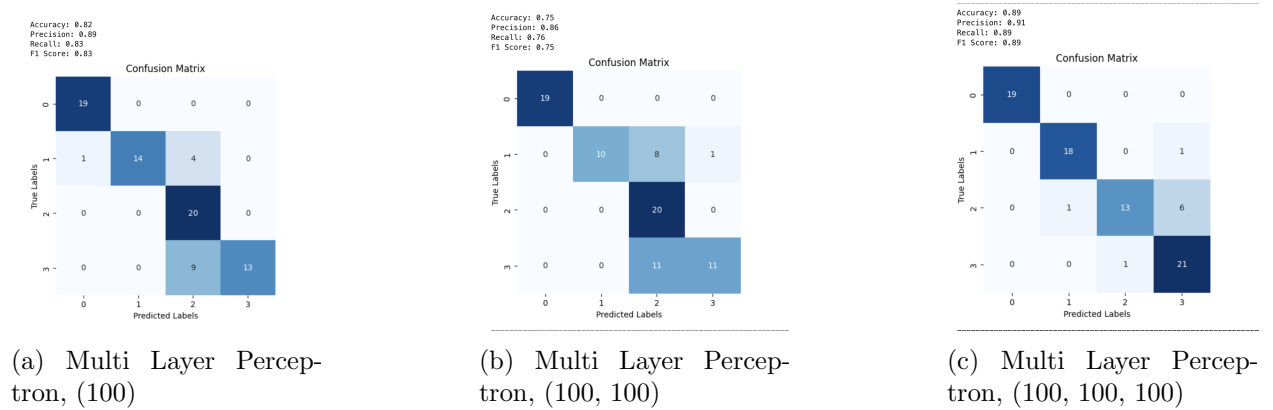
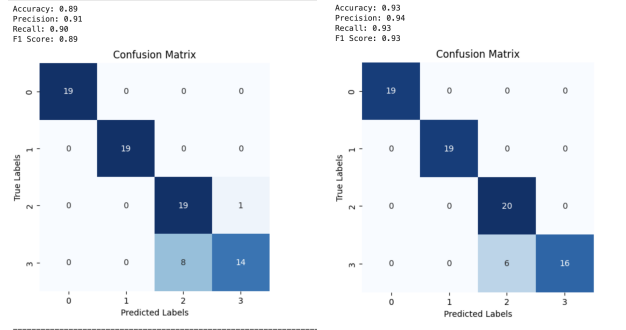


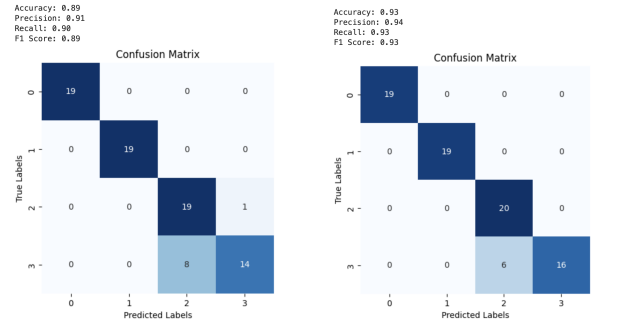
Figure 10: Multi Layer Perceptron Subplots

4.2 HOG Features

Next, the same models are applied to the four sets of HOG features obtained in Task 3. For KNN, $n_neighbors = 4$ is used. The SVC model employs a 'poly' kernel. The MLP is also run with a hidden layer size of 100 and the same settings as before. The results are as below:

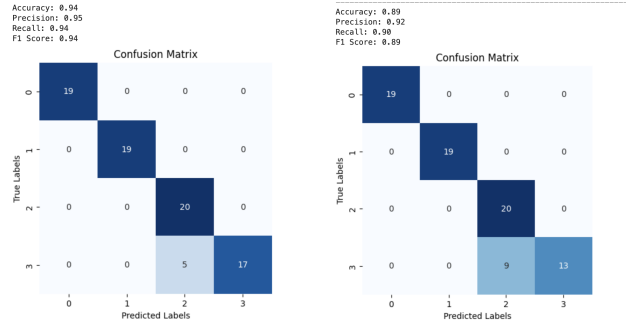


(a) Size: 16, Orientation: 4 (b) Size: 32, Orientation: 4

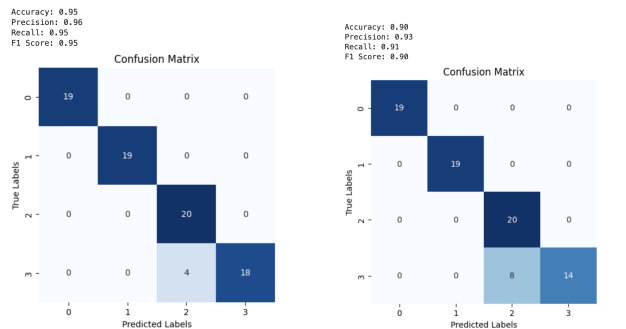


(c) Size: 16, Orientation: 8 (d) Size: 32, Orientation: 8

Figure 11: K-Nearest Neighbours, 4 with different sizes and orientations

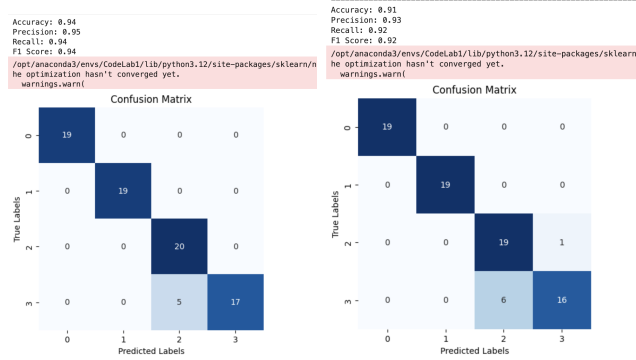


(a) Size: 16, Orientation: 4 (b) Size: 32, Orientation: 4

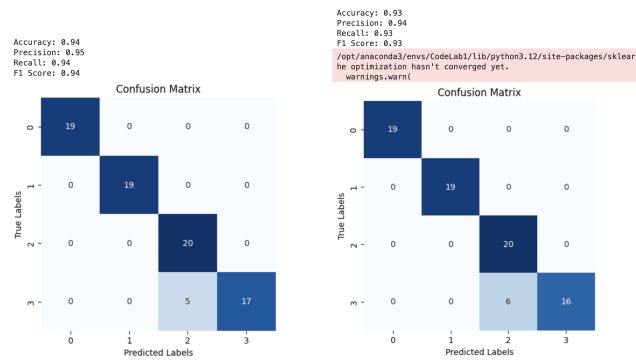


(c) Size: 16, Orientation: 8 (d) Size: 32, Orientation: 8

Figure 12: Support Vector Classifier with kernel=poly and different sizes and orientations



(a) Size: 16, Orientation: 4 (b) Size: 32, Orientation: 4

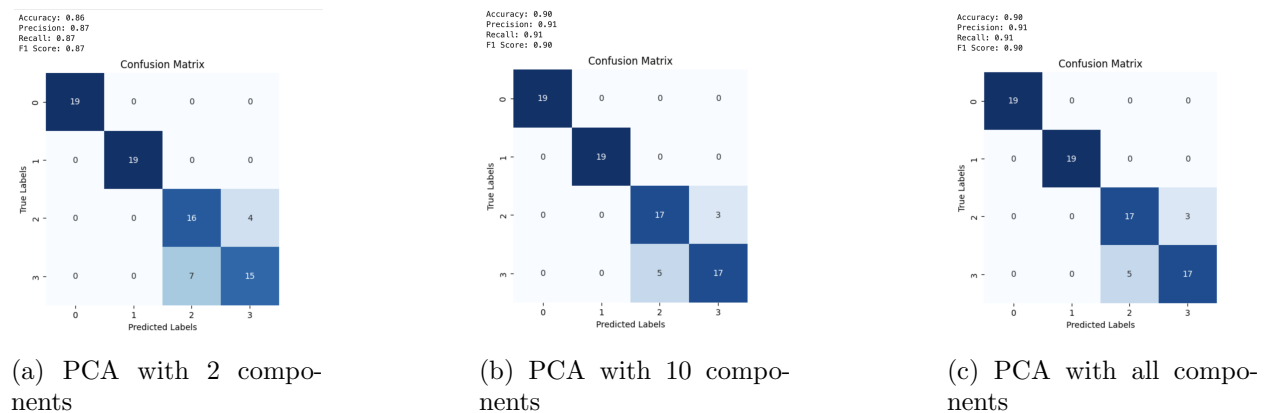


(c) Size: 16, Orientation: 8 (d) Size: 32, Orientation: 8

Figure 13: Multi Layer Perceptron with hidden_layer_size=100 and different sizes and orientations

4.3 PCA-Transformed Features

Finally, PCA is applied to the dataset, and classification is performed on the resulting PCA-transformed features. Similar to the previous steps, KNN, SVC, and MLP are used, where SVC uses a linear kernel. The results for each classification model, including their confusion matrices and performance metrics, are given below:



(a) PCA with 2 components

(b) PCA with 10 components

(c) PCA with all components

Figure 14: KNN: PCA with different components (2, 10, all)

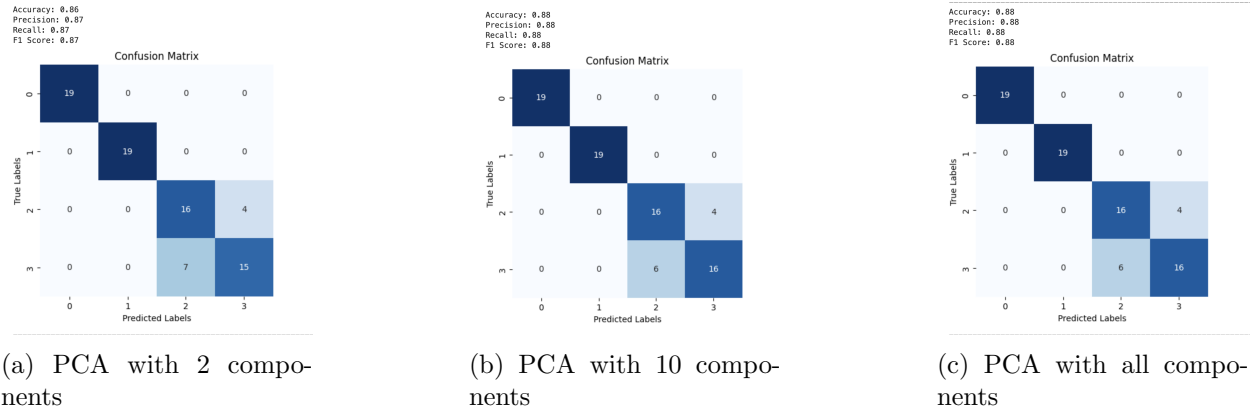


Figure 15: SVC: PCA with different components (2, 10, all)

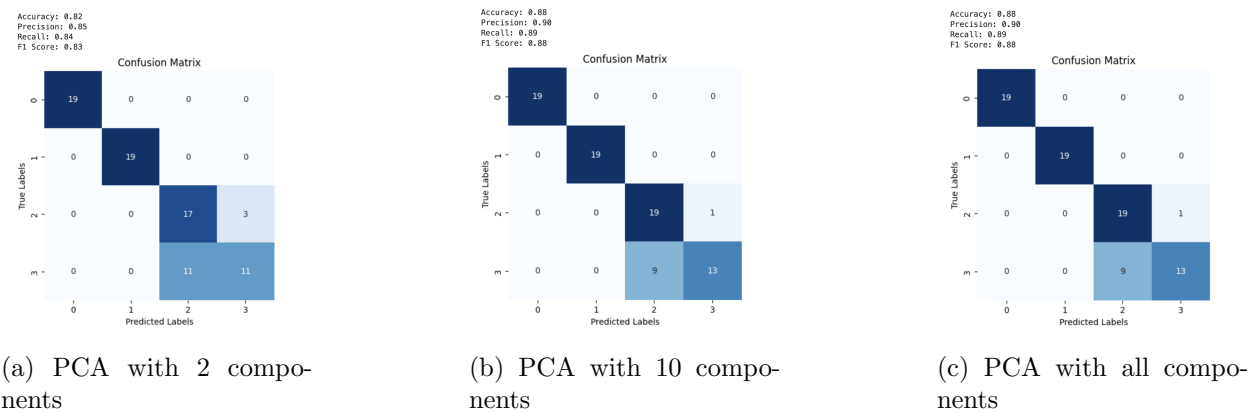


Figure 16: MLP: PCA with different components (2, 10, all)

Question 1

The best-performing classifier for the original dataset is SVC, Poly as it reaches the best scores overall and has a confusion matrix, which shows a higher number of true positives across all classes. The SVC with the Poly kernel captures complex decision boundaries very well and this makes it suitable for this application where there is a non-linear separation.

Question 2

The best-performing classifier for HOG images is SVC, Poly for the Size:16, Orientation:8 one. It has the best score in all of the four metrics. The polynomial kernel, as it was for the original dataset, effectively models the non-linear patterns in the data. As for the Size:16, Orientation:8, with a small cell size of 16 and a higher number of orientations 8, the image is able to capture more detailed gradient information, which then enables the classifier to better distinguish between subtle edge patterns

Question 3

The best-performing classifier for PCA features is KNN where either 10 or all PCA components are used. KNN works well with PCA features because dimensionality reduction already captures the most important variance in the data. After this, KNN's task is easier.

Question 4

PCA with 10 and all components again perform the same. The PCA with 2 components lacks this person very slightly. This observation is only for SVC.

Question 5

For the poly kernel, I might have chosen too small of a max iteration, but I saw that RBF is better than poly.

Question 6

Classes 2 and 3 exhibit lower accuracy, with more false positives, compared to Classes 0 and 1. This pattern is also evident in the PCA plot from Task 3, where there was a significant overlap between Classes 2 and 3 as both of them belonged to chest X-rays. This reflects the difficulty in distinguishing these classes due to their similar features in the original dataset.

Question 7

Having too many layers or neurons can lead to many parameters. This increases the likelihood of the model overfitting the training data by capturing noise instead of generalizing it to new data. This overfitting can result in poorer performance when applied to HOG features, because the model might be struggling when identifying the essential patterns.

5 Bonus Task

The samples belonging to classes 0 and 1 are first discarded, and the values of 0 and 1 are reassigned to classes 2 and 3, respectively. Three classifiers of KNN, SVC, and MLP are trained using a grid search algorithm to identify the best hyperparameters for this problem. The hyperparameter grids for this problem were directly taken from the lab manual.

The identified parameters from the grid search are applied to three different feature sets: normal flattened images, HOG features, and PCA-transformed features. The following results were obtained for each scenario:

```
=== Performing Grid Search for Normal Flattened ===
KNN (Normal Flattened): Best parameters = {'n_neighbors': 3}, Test accuracy = 0.7857142857142857
SVM (Normal Flattened): Best parameters = {'C': 0.1, 'kernel': 'poly', 'max_iter': 300}, Test accuracy = 0.9047619047619048
MLP (Normal Flattened): Best parameters = {'alpha': 0.01, 'hidden_layer_sizes': (100, 100), 'max_iter': 300}, Test accuracy = 0.47619047619047616
```

Figure 17: Trained with normal flattened images.

```
=== Performing Grid Search for HOG 16x4 ===
KNN (HOG 16x4): Best parameters = {'n_neighbors': 3}, Test accuracy = 0.7619047619047619
SVM (HOG 16x4): Best parameters = {'C': 1, 'kernel': 'linear', 'max_iter': 300}, Test accuracy = 0.8809523809523809
MLP (HOG 16x4): Best parameters = {'alpha': 0.01, 'hidden_layer_sizes': (100, 100), 'max_iter': 300}, Test accuracy = 0.8809523809523809
=== Performing Grid Search for HOG 32x4 ===
KNN (HOG 32x4): Best parameters = {'n_neighbors': 15}, Test accuracy = 0.8333333333333334
SVM (HOG 32x4): Best parameters = {'C': 10, 'kernel': 'rbf', 'max_iter': 300}, Test accuracy = 0.8571428571428571
MLP (HOG 32x4): Best parameters = {'alpha': 0.0001, 'hidden_layer_sizes': (100, ), 'max_iter': 300}, Test accuracy = 0.8571428571428571
=== Performing Grid Search for HOG 16x8 ===
KNN (HOG 16x8): Best parameters = {'n_neighbors': 5}, Test accuracy = 0.7857142857142857
SVM (HOG 16x8): Best parameters = {'C': 10, 'kernel': 'rbf', 'max_iter': 300}, Test accuracy = 0.9047619047619048
MLP (HOG 16x8): Best parameters = {'alpha': 0.0001, 'hidden_layer_sizes': (100, ), 'max_iter': 300}, Test accuracy = 0.8809523809523809
=== Performing Grid Search for HOG 32x8 ===
KNN (HOG 32x8): Best parameters = {'n_neighbors': 15}, Test accuracy = 0.8333333333333334
SVM (HOG 32x8): Best parameters = {'C': 10, 'kernel': 'rbf', 'max_iter': 300}, Test accuracy = 0.8571428571428571
MLP (HOG 32x8): Best parameters = {'alpha': 0.001, 'hidden_layer_sizes': (100, ), 'max_iter': 300}, Test accuracy = 0.8571428571428571
```

Figure 18: Trained with HOG.

```

=== Performing Grid Search for PCA 2 components ===
KNN (PCA 2 components): Best parameters = {'n_neighbors': 9}, Test accuracy = 0.7619047619047619
SVM (PCA 2 components): Best parameters = {'C': 10, 'kernel': 'rbf', 'max_iter': 300}, Test accuracy = 0.7380952380952381
MLP (PCA 2 components): Best parameters = {'alpha': 0.0001, 'hidden_layer_sizes': (100, 100), 'max_iter': 300}, Test accuracy = 0.6428571428571429
=== Performing Grid Search for PCA 10 components ===
KNN (PCA 10 components): Best parameters = {'n_neighbors': 5}, Test accuracy = 0.7857142857142857
SVM (PCA 10 components): Best parameters = {'C': 1, 'kernel': 'rbf', 'max_iter': 300}, Test accuracy = 0.8095238095238095
MLP (PCA 10 components): Best parameters = {'alpha': 0.0001, 'hidden_layer_sizes': (100, 100, 100), 'max_iter': 300}, Test accuracy = 0.6904761904761905
=== Performing Grid Search for PCA all components ===
KNN (PCA all components): Best parameters = {'n_neighbors': 5}, Test accuracy = 0.7857142857142857
SVM (PCA all components): Best parameters = {'C': 1, 'kernel': 'rbf', 'max_iter': 300}, Test accuracy = 0.8095238095238095
MLP (PCA all components): Best parameters = {'alpha': 0.001, 'hidden_layer_sizes': (100,), 'max_iter': 300}, Test accuracy = 0.7619047619047619

```

Figure 19: Trained with PCA.

The most suitable model for this CodeLab is the SVC with an RBF kernel trained on HOG features with size:16 and orientation:8. This combination provides the highest accuracy across all datasets and models. This means that the model leverages the detailed gradient information from HOG while its kernel RBF captures non-linear relationships.

One potential feature engineering method to further improve model performance is data augmentation to achieve more diverse images in the datasets. Transformations such as rotations and scaling could help the model generalize better.